Practical Lab: Access Control Models DAC and MAC

Fatima Ezzahra Safar

November 10, 2026

Part I

Discretionary Access Control (DAC)

1 Introduction

In Discretionary Access Control (DAC), the owner of a resource decides who can access it and what they can do with it. In Linux, every file has an owner, a group and three permission types.

The read permission has the symbol r and the value 4. The write permission has the symbol w and the value 2. The execute permission has the symbol x and the value 1.

The permissions are given to three categories: the owner, the group and the others.

2 Guided Experiment

2.1 Step 1: Create a working directory and a file

```
fatima_ezzahra@DESKTOP-MRNJSOT:~$ mkdir ~/DAC_Lab
cd ~/DAC_Lab
touch report.txt
ls -l report.txt
-rw-r--r-- 1 fatima_ezzahra fatima_ezzahra 0 Nov 10 12:24 report.txt
```

Figure 1: Creation of the directory and the file

The file has the default permission `rw-r-r-` (644). The file is empty, so its size is 0.

2.2 Step 2: Change the permissions with chmod 740

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ chmod 740 report.txt
ls -l report.txt
-rwxr----- 1 fatima_ezzahra fatima_ezzahra 0 Nov 10 12:24 report.txt
```

Figure 2: Permissions after chmod 740

The permission 740 means:

- Owner: read, write and execute (4+2+1 = 7).
- Group: read only (4).
- Others: no permissions (0).

2.3 Step 3: Change the owner and the group

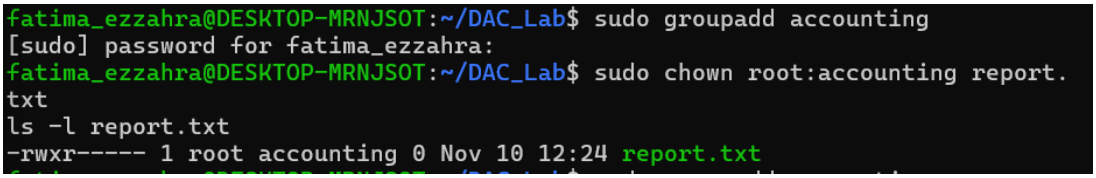


Figure 3: Owner changed to root and group changed to accounting

The command `groupadd` creates the group `accounting`. The command `chown root:accounting` gives the file to the owner `root` and to the group `accounting`. The permissions did not change, but now they apply to different users.

3 Challenge 1: Confidential Documents

The access policy of the company is:

- The owner can read and modify documents.
- Members of the accounting group can read documents.
- Other users have no access.

3.1 Task 1: Create the directory and an example document

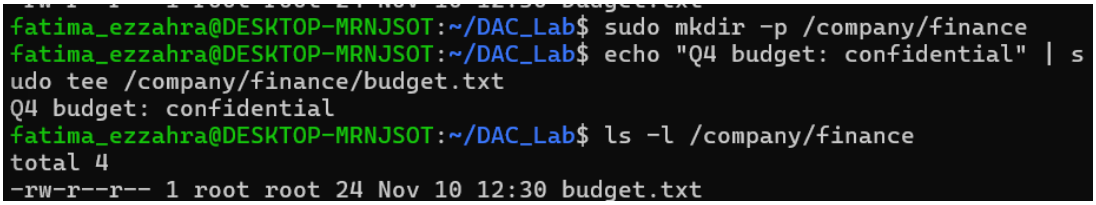


Figure 4: Creation of /company/finance and budget.txt

The option `-p` creates `/company` and `/company/finance` together. I used `sudo tee` to write in the file, because a simple `sudo echo > file` would fail. The redirection is done by my normal user and not by root. At this point the file has the default permission 644, so everybody can read it.

3.2 Task 2: Create the accounting group

The group was already created in the guided experiment, so I only checked that it exists.



Figure 5: The group accounting exists (GID 1002)

When I ran `groupadd accounting` a second time, the system answered that the group already exists. This is not a problem.

3.3 Task 3: Assign the ownership

The owner is `root` and the group is `accounting`. I applied it to the directory and to the file.

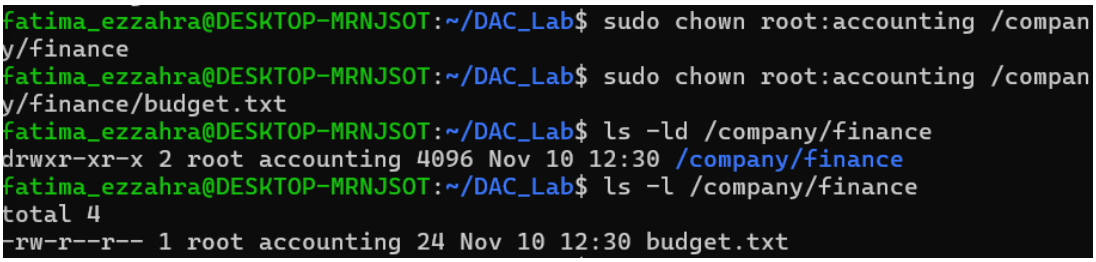


Figure 6: Ownership of the directory and the file

3.4 Task 4: Configure the permissions

From the policy, I calculated the permissions:

- Owner: read and write, `rw-` = 6.
- Group: read only, `r-` = 4.
- Others: no access, `--` = 0.

So the file gets 640. The directory gets 750, because the owner and the group need the execute permission to enter it.

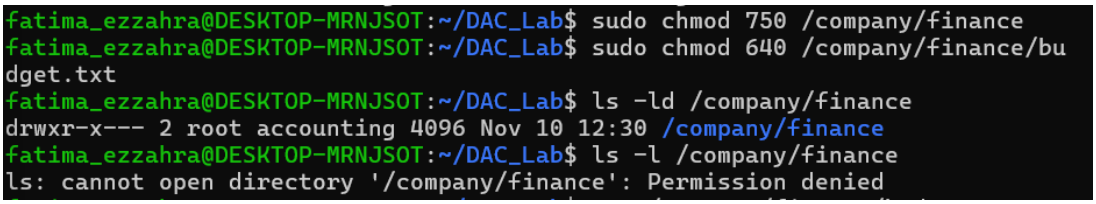


Figure 7: Final permissions of the directory and the file

The directory now shows `drwxr-x--`. The command `ls -l /company/finance` gave `Permission denied`. This is correct: my normal user is not the owner and was not yet in the group, so it is in the category “others”, which has no access.

3.5 Task 5: Verify the final configuration

First, I tested the access with my normal user (category others). Then I added my user to the group accounting and tested as a group member with `sg`.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ cat /company/finance/budget.txt
cat: /company/finance/budget.txt: Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo usermod -aG accounting $USER
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sg accounting -c "cat /company/fin
ance/budget.txt"
Q4 budget: confidential
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sg accounting -c "echo test >> /co
mpany/finance/budget.txt"
sh: 1: cannot create /company/finance/budget.txt: Permission denied
```

Figure 8: Access tests for others and for a member of accounting

The results are the following. An other user who tries to read the file is denied. A member of accounting who tries to read the file is granted access. A member of accounting who tries to write in the file is denied.

The result respects the policy. The group can read but cannot write, and the others have no access.

3.6 Task 6: Directory 750 and file 600

Question: Suppose the directory has permission 750, but a file inside it has permission 600. Can a member of the accounting group read the file?

Answer: No.

Linux checks two levels:

1. The directory has permission 750. The group has `r-x`, so a member of accounting can enter the directory and reach the file.

2. The file has permission 600, which is `rw----`. Only the owner has access. The group has `--`, so it has no read permission.

The permission of the file is the final check. Entering the directory is not enough to read the file, so the group member receives `Permission denied`.

I tested it by setting the file to 600 with `chmod` and then reading it as a member of accounting.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo chmod 600 /company/finance/bu
dget.txt
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ ls -l /company/finance/budget.txt
ls: cannot access '/company/finance/budget.txt': Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sg accounting -c "cat /company/fin
ance/budget.txt"
cat: /company/finance/budget.txt: Permission denied
```

Figure 9: Test with permission 600: the accounting member cannot read the file

The result was `Permission denied`. The command `ls -l` also failed, because my current shell does not have the group accounting yet, so it is treated as “others”. The command `sg` gives the group only for one command, and even with the group the file

with permission 600 stays unreadable. After the test, I restored the correct permission 640 with `chmod`.

This example shows the idea of DAC: the owner of a file can change its permissions at any time with a simple `chmod`, so the owner has the freedom to give or remove access.

Part II
Mandatory Access Control (MAC)

4 Introduction

Mandatory Access Control (MAC) gives access according to a security policy defined by a central authority. Unlike DAC, the owner of a resource cannot freely change this policy.

In this lab, I used a military organization with three classification levels. Public information is accessible to everyone. Confidential information is restricted. Secret information is highly sensitive. Each user has a security clearance, and the access is decided by the policy and not by the owner of the file.

4.1 Bell-LaPadula model

The Bell-LaPadula model protects confidentiality with two rules. The first rule is No Read Up: a user cannot read information above their security clearance. The second rule is No Write Down: a user cannot write information into a lower security classification.

5 Practical Exploration: SELinux

SELinux is a Linux system that applies MAC with security labels and a central policy. I checked its status and displayed the security context of a file.

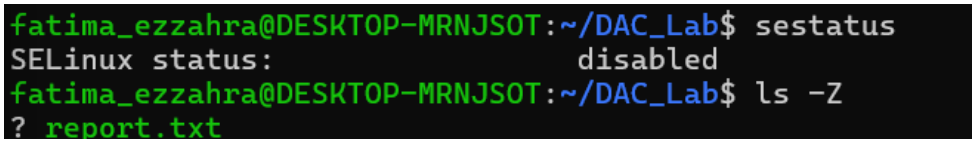


Figure 10: Result of sestatus and ls -Z

The result of `sestatus` is `disabled`, so SELinux is not active on my system. I use Ubuntu under WSL, and this system uses AppArmor by default. The command `ls -Z` shows a question mark before the file name. This means that the file has no SELinux label. On a system with SELinux enabled, this command would show a context with a user, a role, a type and a level.

6 Challenge 2: Military Information System

The organization has three users: Alice has the clearance Secret, Bob has the clearance Confidential and Charlie has the clearance Public. It also has three documents: Document_A is Secret, Document_B is Confidential and Document_C is Public.

6.1 Task 1: Access matrix

I used the No Read Up rule. A user can read a document only if their clearance is equal to or higher than the classification of the document.

The table below shows which documents each user can read.

User (clearance)	Document_A (Secret)	Document_B (Confidential)	Document_C (
Alice (Secret)	Yes	Yes	Yes
Bob (Confidential)	No	Yes	Yes
Charlie (Public)	No	No	Yes

Table 1: Access matrix with the No Read Up rule

6.2 Task 2: Bob tries to read Document_A

Bob has the clearance Confidential and Document_A is classified Secret. Confidential is lower than Secret.

Access denied. The No Read Up rule prevents Bob from reading information above his security clearance.

6.3 Task 3: Alice tries to write into Document_C

Alice has the clearance Secret and Document_C is classified Public. Secret is higher than Public.

Access denied. The No Write Down rule prevents Alice from writing secret information into a lower classification level.

6.4 Task 4: How Bell-LaPadula prevents information leakage

Bell-LaPadula protects confidentiality with two rules: No Read Up and No Write Down. The first rule prevents a user from reading information that is more secret than their clearance. The second rule prevents a user from sending sensitive information to a lower level. Together, these rules prevent unauthorized users from accessing secret information, and they prevent sensitive information from flowing to lower security levels. For example, Alice cannot copy a secret information into Document_C, so Charlie cannot read it.

6.5 Task 5: Clearance is sufficient but the owner denies access

Question: A user has sufficient clearance to read a document, but the owner explicitly denies access with ordinary Linux permissions. Is access necessarily granted under MAC?

Answer: No, access is not necessarily granted.

DAC and MAC are two separate security mechanisms. DAC uses the ordinary Linux permissions, such as `chmod` and `chown`, and the owner decides who can access the file. MAC applies a central security policy, and the owner cannot change it.

On a Linux system with SELinux, both controls can apply at the same time. To access the file, the user must satisfy all the relevant checks: the DAC permissions and the MAC policy. So even if Alice has a sufficient clearance under MAC, the DAC permissions can still deny her access.

Part III
Role-Based Access Control (RBAC)

7 Introduction

Role-Based Access Control (RBAC) gives permissions to roles and not directly to users. A user receives the permissions of the role that he has in the organization. In this lab, the administrator has full access, the developer can read and write, and the viewer can only read. I used Linux groups to simulate the roles.

8 Guided Experiment

8.1 Step 1: Create the roles

I created one group for each role: administrators, developers and viewers.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo groupadd administrators
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo groupadd developers
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo groupadd viewers
```

Figure 11: Creation of the three role groups

8.2 Step 2: Create the users

I created three users: alice, bob and charlie. The command `adduser` asked me for a password and for some personal information, and I filled it in.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo adduser alice
info: Adding user `alice' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `alice' (1006) ...
info: Adding new user `alice' (1006) with group `alice (1006)' ...
info: Creating home directory `/home/alice' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for alice
Enter the new value, or press ENTER for the default
        Full Name []: Alice
        Room Number []: 1006
        Work Phone []: 0688169397
        Home Phone []: 054178587
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `alice' to supplemental / extra groups `users'
info: Adding user `alice' to group `users' ...
```

Figure 12: Creation of the user alice

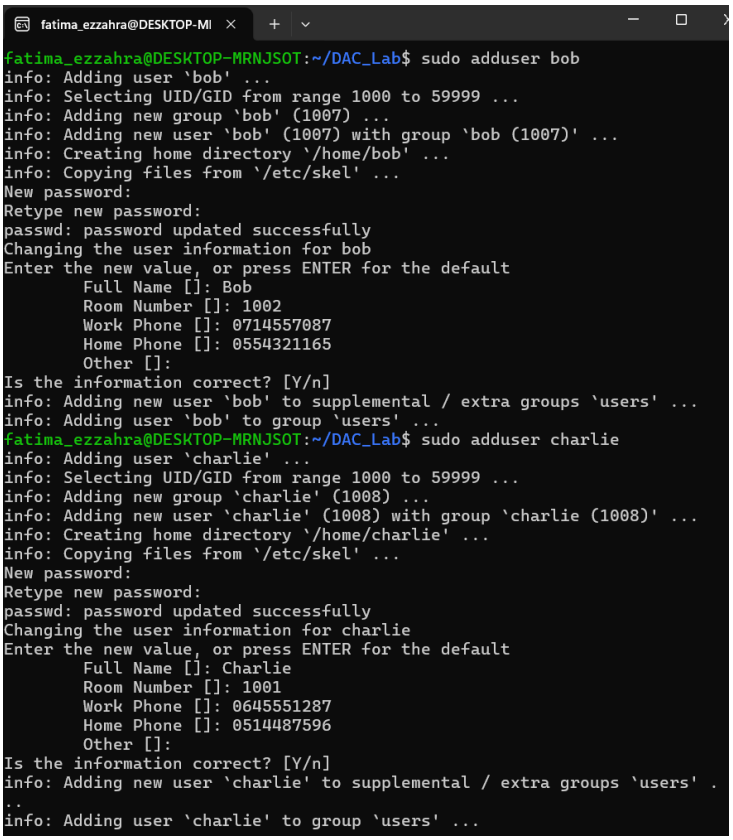


Figure 13: Creation of the users bob and charlie

Then I checked that the three users exist.

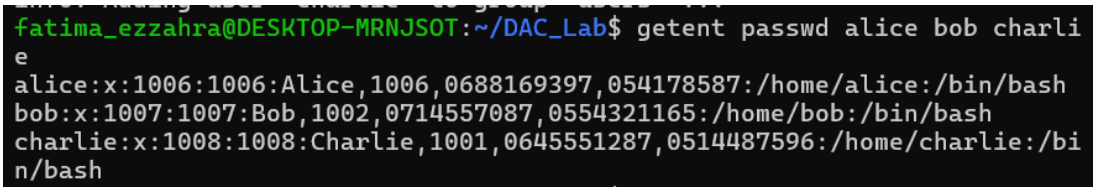


Figure 14: The three users in the system

8.3 Step 3: Assign the roles

I used `usermod -aG` to add each user to a role group. The option `-a` keeps the other groups of the user. Alice became a developer, Bob a viewer and Charlie an administrator.

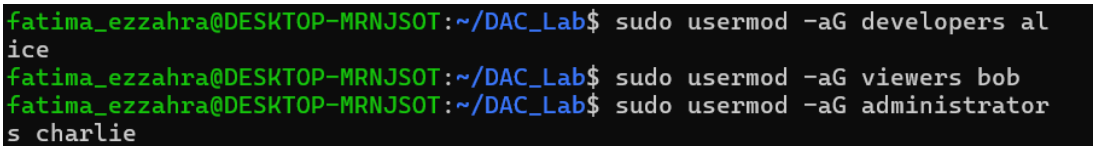


Figure 15: Assignment of the roles

8.4 Step 4: Verify the membership

The command `groups` shows the groups of each user. Each user is in his personal group, in the group `users` (added automatically by `adduser` on my system) and in his role group.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ groups alice
alice : alice users developers
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ groups bob
bob : bob users viewers
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ groups charlie
charlie : charlie users administrators
```

Figure 16: Groups of alice, bob and charlie

8.5 Step 5: Create the organizational directories

I created the directory `/company/source` for the developers with permission 770, and the directory `/company/reports` for the viewers with permission 750.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo mkdir -p /company/source
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo mkdir -p /company/reports

sudo chown root:developers /company/source
sudo chmod 770 /company/source
sudo chown root:viewers /company/reports
sudo chmod 750 /company/reports
```

Figure 17: Creation of the directories and their permissions

9 Challenge 3: Enterprise Access Management

The policy of the company is the following. A developer can read and write the source code and can read the reports. A viewer has no access to the source code and can read the reports. An administrator has full access to the source code and to the reports.

9.1 Task 1: Create the groups and assign the users

The groups and the users were already created in the guided experiment, so I only checked them. The group developers contains alice, the group viewers contains bob and the group administrators contains charlie.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ getent group developers viewer
s administrators
developers:x:1004:alice
viewers:x:1005:bob
administrators:x:1003:charlie
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ groups alice bob charlie
alice : alice users developers
bob : bob users viewers
charlie : charlie users administrators
```

Figure 18: Check of the groups and of the members

9.2 Task 2: Configure the permissions for both directories

I used the same configuration as in the guided experiment. The directory `/company/source` belongs to the group developers with permission 770. The directory `/company/reports` belongs to the group viewers with permission 750.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo chown root:developers /co
mpany/source
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo chmod 770 /company/source
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo chown root:viewers /compa
ny/reports
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo chmod 750 /company/report
s
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ ls -ld /company/source /compan
y/reports
drwxr-x--- 2 root viewers    4096 Nov 10 13:18 /company/reports
drwxrwx--- 2 root developers 4096 Nov 10 13:18 /company/source
```

Figure 19: Permissions of the two directories

9.3 Task 3: Does the configuration satisfy the complete policy?

I tested each user on each directory with `sudo -u`. Alice can use the directory source, but she is denied on the directory reports. Bob is denied on source and he can read reports. Charlie is denied on both directories.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u bob ls /company/source

ls: cannot open directory '/company/source': Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u alice ls /company/repo
rts
ls: cannot open directory '/company/reports': Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u bob ls /company/source
ls: cannot open directory '/company/source': Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u bob ls /company/report
s
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u charlie ls /company/so
urce
ls: cannot open directory '/company/source': Permission denied
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo -u charlie ls /company/re
ports
ls: cannot open directory '/company/reports': Permission denied
```

Figure 20: Access tests before the correction

Answer: No, the configuration does not satisfy the complete policy. Two permissions are missing:

- The developers cannot read the directory reports, because its group is viewers and Alice is not in this group.
- The administrators have no access to source and reports. Charlie is not the owner and he is not in the groups developers and viewers, so he is treated as "others", and others have no permission.

9.4 Task 4: Correct the missing permissions

A directory has only one group, so I used ACLs (Access Control Lists) to add permissions for other groups. I gave the group developers the permission `r-x` on reports, and I gave the group administrators the permission `rwx` on source and on reports. Then I displayed the ACLs with `getfacl`.

```
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo apt install -y acl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
acl is already the newest version (2.3.2-1build1.1).
acl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo setfacl -m g:developers:r
-x /company/reports
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo setfacl -m g:administrato
rs:rwx /company/source
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ sudo setfacl -m g:administrato
rs:rwx /company/reports
fatima_ezzahra@DESKTOP-MRNJSOT:~/DAC_Lab$ getfacl /company/source /compa
ny/reports
getfacl: Removing leading '/' from absolute path names
# file: company/source
# owner: root
# group: developers
user::rwx
group::rwx
group:administrators:rwx
mask::rwx
other::---

# file: company/reports
# owner: root
# group: viewers
user::rwx
group::r-x
group:administrators:rwx
group:developers:r-x
mask::rwx
other::---
```

Figure 21: Addition of the ACL permissions

Then I tested the final policy. Alice can read the directory reports, but she cannot create a file in it. Alice can create a file in source. Charlie can create files in source and in reports. Bob is still denied on source. The command `ls -l` from my own user was denied, because my user is not in these groups. At the end, I removed the test files.

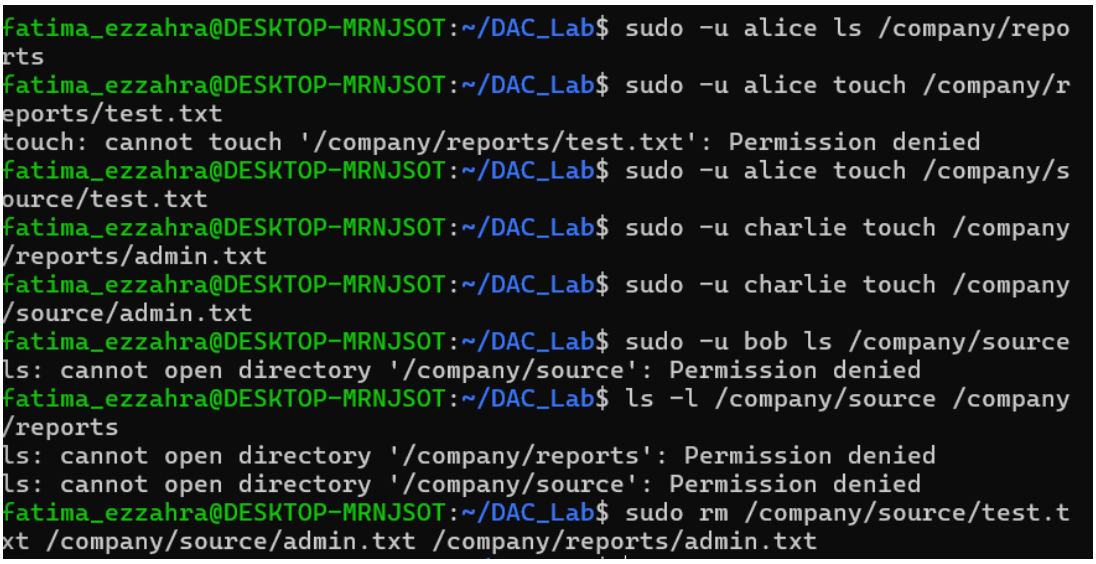


Figure 22: Access tests after the correction

Now the configuration satisfies the complete policy.

Part IV
Attribute-Based Access Control (ABAC)

10 Introduction

Attribute-Based Access Control (ABAC) makes the access decision with attributes. These attributes can describe the subject (the user, for example his department and his clearance), the object (the resource, for example its classification), the action (read or write) and the environment (the context, for example the time).

11 Challenge 4: Banking Access Policy

A bank applies this policy. The department of the employee must be the same as the department of the document. The clearance of the employee must be sufficient. The access is allowed only between 08:00 and 18:00. If one condition is false, the access is denied.

11.1 Task 1: Evaluate the access requests

I checked the three conditions for each request. The table shows the decision for each user.

User	Decision
Alice	GRANTED
Bob	DENIED
Charlie	DENIED
David	DENIED

Table 2: Decision for each access request

11.2 Task 2: Attribute responsible for each denial

The attribute responsible for the denial of Bob is the department. The attribute responsible for the denial of Charlie is the clearance. The attribute responsible for the denial of David is the time. The department and the clearance are attributes of the subject, and the time is an attribute of the environment. Alice has no denial, because her request was granted.

11.3 Task 3: Alice requests access to an HR document

Alice belongs to the Finance department, but the document belongs to the HR department. So Finance is not equal to HR.

Access denied. The reason is the department mismatch. Even if Alice has the clearance Secret and asks during the allowed hours, she does not satisfy the department condition.

11.4 Task 4: David requests access at 17:00

Question: Does the decision change?

Answer: Yes, the decision changes. At 17:00, the department is the same (Finance and Finance), the clearance is enough (Secret is equal to Secret), and the time is inside the allowed hours (08:00 is before 17:00, and 17:00 is before 18:00). David satisfies the three conditions, so the access is **granted**. The decision changes because the only condition that failed, the time, is now satisfied.

11.5 Task 5: Why is ABAC more flexible than RBAC?

RBAC grants access with predefined roles, while ABAC grants access with many attributes, such as the department, the clearance, the classification of the resource and the time. ABAC is more flexible because it can consider the context of each access request without creating many different roles. For example, with RBAC the bank would need a different role for each department, each clearance level and each period of time, but with ABAC one policy is enough.